Collaboration to Repository-Level Vulnerability Detection

Xin-Cheng Wen
Harbin Institute of Technology
Shenzhen, China
xiamenwxc@foxmail.com

Abstract

Large Language Model (LLM)-based methods have proven to be effective for many software engineering domains, with a potential for substantial productivity effective for software vulnerability detection. However, due to the limitation of the length of input contexts of LLM, the existing LLM-based methods mainly focus on detecting function-level and leveraging the in-file context information for vulnerability detection (i.e., intra-procedural vulnerabilities), ignoring the more complex inter-procedural vulnerability detection scenarios in practice. For instance, in real-world scenarios, developers routinely engage with program analysis to detect vulnerabilities that span multiple cross-file information within repositories.

Since complex processes tend to have redundancy dependencies from spanning multiple files in the repository level and invoking multiple static analysis tools, the ideal goal of vulnerability detection is to extract the vulnerability-related information from the repository and provide potential possible explanations for vulnerability triggers. However, such a goal is hard to achieve, and thus in this work, we design three works through multi-agent collaboration to approach the goal of repository-level vulnerability detection.

CCS Concepts

• **Software and its engineering** → *Software development techniques.*

Keywords

Software Vulnerability Detection, Large Language Model

ACM Reference Format:

Xin-Cheng Wen. 2024. Collaboration to Repository-Level Vulnerability Detection. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24), September 16–20, 2024, Vienna, Austria.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3650212.3685562

1 Introduction

Software vulnerabilities are widely caused by insecure source code and can be exploited by attackers to attack software systems, leading to a wide array of security issues such as data breaches, identity theft and loss of service. In recent years, the growing scale and complexity of software have exacerbated the impact of software vulnerability detection, potentially resulting in substantial societal losses. For instance, in 2023, Cisco identified a CVE-2023-20198 [1] vulnerability in the WebUI, allowing unauthorized remote attackers to gain elevated privileges. To date, over 41,000 affected devices have been compromised, causing considerable losses for enterprises. Therefore, it is crucial to automate software vulnerability detection.

Recently, many Deep Learning (DL)-based approaches have been proposed for automated software vulnerability detection. These methods provide a broader range of capabilities in identifying diverse software vulnerabilities, which have emerged as effective solutions for vulnerability detection [4]. They extract multiple types of representations from source code and leverage DL models to detect a variety of software vulnerabilities. For example, LineVul [7] uses the BPE tokenizers and adopts the pre-trained model CodeBERT to learn the vulnerability representation. However, based on the existing approaches, software vulnerability detection still suffers from the following challenges:

(1) Ignoring the complex inter-procedural vulnerability detection. In real-world scenarios, vulnerabilities often involve dependencies between multiple files and functions within a repository, while individual functions may not exhibit vulnerabilities in isolation. Current vulnerability detection methods primarily focus on function-level or single-file granularity, excelling at intra-function vulnerability scanning but neglecting inter-procedural information (e.g., control flow graph [11], and data flow graph [19]). Consequently, DL models trained (e.g., supervised fine-tuning, prompt tuning, instruct tuning [17, 20]) without considering inter-procedural information are biased, overlooking inter-procedural vulnerabilities and thereby limiting their effectiveness in practical applications.

(2) High false positive rate. Existing DL-based software vulnerability detection methods commonly suffer from a high false positive rate. This challenge arises because current models fail to adequately comprehend the triggers of vulnerabilities or lack the dependency information of a given source code. Therefore, invoking static code analysis with Large Language Models (LLMs) to enhance the understanding of vulnerability triggers and improve vulnerability detection is of significant research value, as it can generate higher-quality results and avoid the high false positive rates.

To address the above challenges in vulnerability detection, in this work, we aim to propose an integrated process to detect intra- and inter-procedure vulnerability at the repository level. The ideal goal of vulnerability detection is to extract the vulnerability-triggering execution path from the repository and provide potential possible explanations for both intra- and inter-procedural vulnerabilities for the whole repository. However, such a goal is hard to achieve, and thus in this work, we plan to collect the repository-level information, evaluate the repository-level vulnerability detection, and plan the whole process by a multi-agent system to help us approach the goal.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISSTA '24, September 16–20, 2024, Vienna, Austria
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0612-7/24/09
https://doi.org/10.1145/3650212.3685562

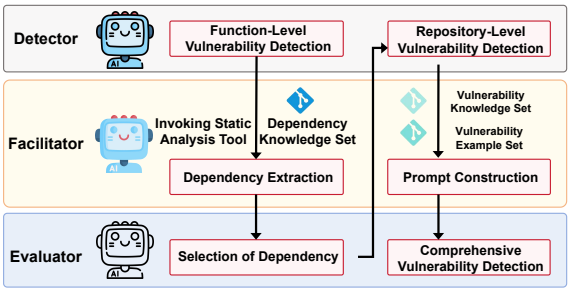


Figure 1: The example of multi-agents collaboration.

2 Background

2.1 Problem Formulation

We denote the dataset with training instances collected from the real world as $D = \{\mathcal{X}, \mathcal{Y}, \mathcal{Z}\}$, where X, $\mathcal{Y}$ and $\mathcal{Z}$ denotes the raw code snippet, vulnerability label, and the whole repository-level source code, respectively. For each raw code snippet $\mathcal{X}$, we need to invoke the static analysis techniques f to extract the potential "useful" information from the repository $\mathcal{Z}$. Subsequently, employing the prompts or instruction construction techniques g to enable the model to ascertain whether the constructed input $\mathcal{I}$ contains vulnerabilities $\mathcal{Y}$ and generate the reasons $\mathcal{A}$ to explain the vulnerability-triggering paths .

$$f : \{\mathcal{X}, \mathcal{Z}\} \mapsto \mathcal{I}, \quad g : \mathcal{I} \mapsto \{\mathcal{Y}, \mathcal{A}\} \tag{1}$$

2.2 Related Work

The existing DL-vulnerability detection methods can be classified into discriminative-based and generative-based methods. Discriminative-based methods [2, 5, 9, 18] typically have the feature extraction process, followed by a training classifier to predict whether each code snippet contains a vulnerability (denoted as 0 for non-vulnerability and 1 for vulnerability). For instance, SVulD [15] utilizes contrastive learning techniques to learn distinguishing semantic representations and fine-tunes the classifier for vulnerability detection. Generative-based methods [3, 16] are the recently widely-used technique, which has seen widespread adoption in natural language processing [13] and software engineering [10], which frequently employ various prompts and chain-of-thought techniques to incorporate vulnerability domain knowledge [8]. For example, GRACE [14] leverages graph structures and in-context learning to identify vulnerabilities effectively.

3 Research Proposal

To address the serious efficiency challenges of repository-level vulnerability detection, there are two research questions as follows:

- How to evaluate the repository-level vulnerability detection?
- How to effectively detect the intra- and inter-procedural vulnerability simultaneously?

To address the first research question, we plan to investigate the performance of existing generative large language models in repository-level vulnerability detection. Specifically, we will first construct a repository-level vulnerability benchmark. Subsequently, we will develop an evaluation system to assess the effectiveness of current LLM methods in detecting repository-level vulnerabilities. To tackle the second research question we intend to employ a multi-agent approach to determine which repository-level dependencies are more effective for vulnerability detection. Additionally, it is essential to develop effective prompting methods that incorporate domain-specific information about vulnerabilities. Therefore, we design the following three works.

The first work aims to build a repository-level dataset as a foundation for my work in the area of repository-level vulnerability detection. We plan to collect a large-scale repository-level vulnerability dataset, which provides repository-level information for each vulnerability patch. On the one hand, it offers the potential for comprehensive warehouse-level vulnerability detection. On the other hand, it enhances my understanding of the domain knowledge pertinent to software vulnerabilities.

The second work aims to build an evaluation system for exploring whether LLMs and techniques facilitate repository-level vulnerability detection. As discussed in Section 1, existing methods do not address vulnerability detection at the repository level. Therefore, we propose developing an evaluation system that leverages various repository-level information (e.g., function call) to evaluate the effectiveness of current vulnerability detection methods at both the function- and repository- levels. Our approach plans to explore multiple types of baselines, encompassing both discriminative-based and generative-based methods.

The third work aims to explore how the selection of repository level dependencies and the construction of various types of inputs contribute to repository-level vulnerability detection. During the first and second steps, we explore the potential of evaluating repository-level vulnerability detection by addressing the initial research question. In this work, we plan to propose a multi-agent collaboration framework encompassing various roles, including detector, facilitator, and evaluator. Specifically, as shown in Figure 1, the detector will be responsible for identifying vulnerabilities in the given code. The facilitator will invoke external tools (e.g., Tree-sitter [6], Checkmarx [12]) to reduce the false positive rate and integrate domain knowledge. The evaluator assesses the actions of other role agents, such as evaluating the compliance of selected dependencies or providing explanations. Finally, we will conduct a comprehensive evaluation.

4 Conclusion

The ideal goal of repository-level vulnerability detection is to extract the vulnerability-related information from the repository and provide potential possible explanations for vulnerability triggers. In my future research, we plan to design three works to approach the goal through a collaborative multi-agent system. The three works include collecting a large-scale repository-level dataset, conducting an evaluation system, and proposing a final multi-agent-based system for repository-level vulnerability detection.

Acknowledgments

Xin-Cheng is advised by Prof. Cuiyun Gao. Research is supported by National Key R&D Program of China (No.2022YFB3103900).

References

[1] 2024. CVE-2023-20198 Detail. https://nvd.nist.gov/vuln/detail/CVE-2023-20198

[2] Sicong Cao, Xiaobing Sun, Lili Bo, Ying Wei, and Bin Li. 2021. *BGNN4VD*: Constructing Bidirectional Graph Neural-Network for Vulnerability Detection. *Inf. Softw. Technol.* 136 (2021), 106576.

[3] ChatGPT. 2022. ChatGPT. https://chat.openai.com/.

[4] Xiao Cheng, Xu Nie, Ningke Li, Haoyu Wangand Zheng Zheng, and Yulei Sui. 2022. How About Bug-Triggering Paths?-Understanding and Characterizing Learning-Based Vulnerability Detectors. *IEEE Transactions on Dependable and Secure Computing*.

[5] Xu Duan, Jingzheng Wu, Shouling Ji, Zhiqing Rui, Tianyue Luo, Mutian Yang, and Yanjun Wu. 2019. VulSniper: Focus Your Attention to Shoot Fine-Grained Vulnerabilities. In *IJCAI*. ijcai.org, 4665–4671.

[6] Coraline Ada Ehmke. 2023. Tree-sitter. https://tree-sitter.github.io/tree-sitter/

[7] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In *MSR*. ACM, 608–620.

[8] Michael Fu, Chakkrit Tantithamthavorn, Van Nguyen, and Trung Le. 2023. ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We? *CoRR* abs/2310.09810 (2023).

[9] David Hin, Andrey Kan, Huaming Chen, and Muhammad Ali Babar. 2022. LineVD: Statement-level Vulnerability Detection using Graph Neural Networks. In *MSR*. ACM, 596–607.

[10] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John C. Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *CoRR* abs/2308.10620 (2023).

[11] Xuan Huo, Ming Li, and Zhi-Hua Zhou. 2020. Control Flow Graph Embedding Based on Multi-Instance Decomposition for Bug Localization. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 4223–4230.

[12] Israel. [n.d.]. Checkmarx. https://www.checkmarx.com/.

[13] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What Makes Good In-Context Examples for GPT-3?. In *Proceedings of Deep Learning Inside Out: The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, DeeLIO@ACL 2022, Dublin, Ireland and Online, May 27, 2022*. Association for Computational Linguistics, 100–114.

[14] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *J. Syst. Softw.* 212 (2024), 112031.

[15] Chao Ni, Xin Yin, Kaiwen Yang, Dehai Zhao, Zhenchang Xing, and Xin Xia. 2023. Distinguishing Look-Alike Innocent and Vulnerable Code by Subtle Semantic Representation Learning and Explanation. *CoRR* abs/2308.11237 (2023).

[16] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023).

[17] Nima Tajbakhsh, Jae Y. Shin, Suryakanth R. Gurudu, R. Todd Hurst, Christopher B. Kendall, Michael B. Gotway, and Jianming Liang. 2016. Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning? *IEEE Trans. Medical Imaging* 35, 5 (2016), 1299–1312.

[18] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, and Zheng Wang. 2021. Combining Graph-Based Learning With Automated Data Collection for Code Vulnerability Detection. *IEEE Trans. Inf. Forensics Secur.* 16 (2021), 1943–1958.

[19] Xiaomeng Wang, Tao Zhang, Runpu Wu, Wei Xin, and Changyu Hou. 2018. CPGVA: Code Property Graph based Vulnerability Analysis by Deep Learning. In *10th International Conference on Advanced Infocomm Technology, ICAIT 2018, Stockholm, Sweden, August 12-15, 2018*. IEEE, 184–188.

[20] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.

Received 2024-07-07; accepted 2024-07-22